



TREINA RECIFE
DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

Linguagem SQL

Structured Query Language – Parte 01



Quem sou eu?

CEO and Founder da Empresa Treina Recife
Mestre em Administração – UFPE
Especialista em Gestão da TIC
Especialista em Análise e Desenvolvimento de Sistemas
Graduado em Matemática
Consultor Sênior na Gestão Organizacional e de TI
Instrutor Cursos Executivos
Professor de Pós-Graduação na FAFIRE, UNICAP e no SENAI-PE



Prof. ME Rogério Aguiar C. Teixeira



@prof_rogerio.aguiar



+55 81 9.9247-4667



rogerio.aguiar@treinarecife.com.br



<http://lattes.cnpq.br/1025838649183053>



<http://br.linkedin.com/pub/rogerio-aguiar-teixeira/23/634/28a>



O que vamos Aprender?

01 - Introdução a Bancos de Dados

02 - MySQL Server e o Workbench

03 - Conhecendo um modelo de dados

04 – **DDL**

(Data Definition Language)
Linguagem de **Definição** de Dados

05 - **DML**

(Data Manipulation Language)
Linguagem de **Manipulação** de Dados

06 - **DQL**

(Data Query Language)
Linguagem de **Consulta** de Dados



← → ↻ w3schools.com/sql/sql_ref_keywords.asp

HTML CSS JAVASCRIPT **SQL** PYTHON PHP BOOTSTRAP HOW TO W3.CSS JAVA JQUERY C C++ C# R React

SQL Tutorial

SQL HOME

SQL Intro

SQL Syntax

SQL Select

SQL Select Distinct

SQL Where

SQL And, Or, Not

SQL Order By

SQL Insert Into

SQL Null Values

SQL Update

SQL Delete

SQL Select Top

SQL Min and Max

SQL Count, Avg, Sum

SQL Like

SQL Wildcards

SQL In

SQL Between

SQL Aliases

SQL Joins

SQL Keywords Reference

< Previous

Next >

This SQL keywords reference contains the reserved words in SQL.

SQL Keywords

Keyword	Description
<u>ADD</u>	Adds a column in an existing table
<u>ADD CONSTRAINT</u>	Adds a constraint after a table is already created
<u>ALL</u>	Returns true if all of the subquery values meet the condition
<u>ALTER</u>	Adds, deletes, or modifies columns in a table, or changes the data type of a column in a table
<u>ALTER COLUMN</u>	Changes the data type of a column in a table
<u>ALTER TABLE</u>	Adds, deletes, or modifies columns in a table
<u>AND</u>	Only includes rows where both conditions is true

https://www.w3schools.com/sql/sql_ref_keywords.asp



Referencial Teórico



www.treinarecife.com.br



Referencial Teórico



Guia de referência

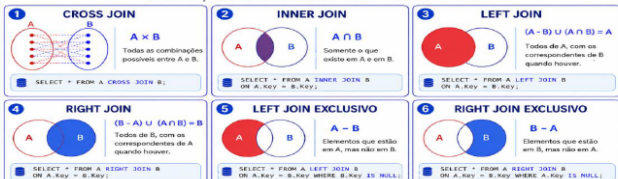
CURSO BANCO DE DADOS - SQL

Professor: Rogério Aguiar Teixeira, MsC

criar e destruir banco de dados	
<pre>CREATE DATABASE [IF NOT EXISTS] nome_bd DROP DATABASE [IF EXISTS] nome_bd</pre>	
criando tabelas	
<pre>CREATE TABLE [IF NOT EXISTS] nome_tabela (nome_coluna1 tipo[signed unsigned][not null null][default Valor][auto_increment] [comment 'texto'], PRIMARY KEY (coluna1, coluna2, ...), FOREIGN KEY (coluna) REFERENCES nome_tabela1 (coluna_tabela1); DROP TABLE [IF EXISTS] nome_tabela;</pre>	
ADICIONANDO COLUNAS EM TABELAS	
<pre>ALTER TABLE table_name ADD column_name datatype;</pre>	
ALTERANDO COLUNAS DE TABELAS	
<pre>ALTER TABLE table_name MODIFY COLUMN column_name1 datatype, ...; ALTER TABLE table_name CHANGE COLUMN column_name1 novo_nome tipo;</pre>	
REMOVENDO COLUNAS DE TABELAS	
<pre>ALTER TABLE table_name DROP COLUMN column_name; criar e destruir a PK - APOS A CRIAÇÃO DA TABELA ALTER TABLE nome_tabela ADD PRIMARY KEY (nome_coluna); ALTER TABLE nome_tabela DROP PRIMARY KEY;</pre>	
criar e destruir a FK - APOS A CRIAÇÃO DA TABELA	
<pre>ALTER TABLE nome_tabela1 ADD FOREIGN KEY (nome_coluna1) REFERENCES nome_tabela2 (nome_coluna2); ALTER TABLE dependentes DROP INDEX dependentes_idx1;</pre>	
EXIBIR COMO A TABELA FOI CRIADA	
<pre>SHOW CREATE TABLE nome_tabela;</pre>	
criar e destruir índices para uma tabela	
<pre>CREATE [UNIQUE] INDEX index_name ON table_name (column1, column2, ...); DROP INDEX index_name ON table_name;</pre>	
UPDATE	
<pre>INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3,...); INSERT INTO table_name VALUES (value1, value2, value3, ...);</pre>	
DELETE	
<pre>UPDATE table_name SET column1 = value1, column2 = value2, ... [WHERE condition]; DELETE FROM table_name [WHERE condition];</pre>	
SELECT	
<pre>SELECT column1, column2, ... FROM table_name [WHERE condition AND/OR condition] [GROUP BY column_name(s)] [HAVING condition] [ORDER BY column1, column2, ... ASC DESC] [LIMIT number];</pre>	
FUNÇÕES	
<pre>MAX(): MIN(): COUNT(): AVG(): SUM(): CURRENT_DATE: DATE_FORMAT('2025-09-07', '%d/%m/%Y'); DAY(): MONTH(), YEAR() BETWEEN X AND Y: LIKE: IN: OR: AND: : LENGTH(): AS</pre>	

JOINS SQL e Teoria dos Conjuntos

Como interpretar os relacionamentos entre Tabela A e Tabela B



Guia de referência

CURSO BANCO DE DADOS - SQL

Professor: Rogério Aguiar Teixeira, MsC

Parametros utilizadas nas colunas do Select

Operador	Descrição	Exemplo	Efeito sobre
Concat	Concatenação	Select concat(nome, telefone, ...)	Concatena colunas a outras colunas
AS	Alias - Renomeia um cabeçalho de coluna	Select nome as 'Nome da Pessoa'	Gera um apelido para a coluna, a palavra-chave AS é opcional
Length	Comprimento do conteúdo de uma coluna	Select nome, length(nome)	Retorna o nome e seu comprimento considera letras acentuadas como dois caracteres
Char_length()		Select nome, char_length(nome)	Retorna a qtd exata das letras (não é padrão)
Distinct	Linhas distintas	Select distinct nome, telefone	Se houver linhas duplicadas traz apenas uma ocorrência dela.
Operações	Operações Aritméticas	Select col+colB	

Clausula WHERE (Condição)

Operador	Descrição	Exemplo	Efeito sobre
=	Igual	WHERE cpf = 111;	Linhas cujo cpf for igual a 111
>	Maior que	WHERE cpf > 222;	Linhas cujo cpf for maior que 222
<	Menor que	WHERE cpf < 222;	Linhas cujo cpf for menor que 222
>=	Maior ou igual		
<=	Menor ou igual	WHERE cpf >= 222;	Linhas cujo cpf for maior ou igual que 222
<> ou !=	Diferente	WHERE cpf != 222;	Linhas cujo cpf for diferente de 222
BETWEEN	Entre uma faixa de valor	WHERE cpf BETWEEN 111 AND 333; OR cpf >= 111 and cpf <= 333;	Linhas entre cujo cpf esteja entre os valores 111 e 333, incluí os valores 111 e nem o 333
LIKE	Pesquisa por parte, tanto para campos numéricos quanto textos.	WHERE nome LIKE 'C%'; WHERE nome LIKE 'A%'; WHERE nome LIKE '%A%'; WHERE cpf LIKE '1%';	Linha cujo nome inicie pela letra "C" ou "c" Linha cujo nome termine com a letra "A,a,i,j,k" Linha cujo nome contenha a letra "A,a,i,j,k" Linha cujo nome possua a letra "a" na segunda posição Linha cujo cpf possua três posições com a segunda = 1
IN	Múltiplos valores	WHERE cpf IN (111,333);	Linha cujo cpf seja igual a 111 ou 333;
NOT IN	Múltiplos valores	WHERE cpf NOT IN (111,333);	Linha cujo cpf seja diferente de 111 e 333
OR	Múltiplas condições	WHERE cpf = 111 OR cpf = 333	Linha cujo cpf seja igual a 111 ou 333
AND	Múltiplas condições	WHERE cpf = 111 AND nome = 'Rogério Aguiar'	Linha cujo cpf seja igual a 111 e nome igual a "Rogério Aguiar"

Manipulação da Datas

Função	Comando	Exemplo
Data Corrente	CURRENT_DATE	2022-09-07
Formatar Data	DATE_FORMAT('2022-09-07', '%d/%m/%Y')	07/09/2022
Dia	DAY(campo_data)	DAY('2022-09-07') → 7
Mês	MONTH(campo_data)	MONTH('2022-09-07') → 9
Ano	YEAR(campo_data)	YEAR('2022-09-07') → 2022



Guia de referência

CURSO BANCO DE DADOS - SQL

Professor: Rogério Aguiar Teixeira, MsC

Principais tipos de campos no MySQL

Campos Numéricos

Alguns campos numéricos possuem a opção UNSIGNED. Isso quer dizer que o número não pode ser negativo. Por exemplo: o campo TINYINT com a opção UNSIGNED vai de 0 até 255, sem a opção UNSIGNED vai de -128 até 127.

A opção ZEROFILL completa o campo com zeros. Por exemplo: se você cadastrar '65' em um campo INT(5) - ou seja, inteiro com 5 dígitos - com a opção ZEROFILL habilitada, o MySQL irá cadastrar '00065'. Sem a opção ZEROFILL habilitada, o MySQL irá cadastrar '65'.

- TINYINT:** Um número muito pequeno, de 0 até 255 (UNSIGNED) ou -128 até 127. Ocupa 1 byte.
- SMALLINT:** Um inteiro pequeno, de 0 a 65535 (UNSIGNED) ou -32768 até 32767. Ocupa 2 bytes.
- MEDIUMINT:** Um inteiro de tamanho médio, de 0 a 16777215 (UNSIGNED) ou -8388608 a 8388607. Ocupa 3 bytes.
- INT ou INTEGER:** Um inteiro de tamanho normal. De 0 a 4294967295 (UNSIGNED) ou -2147483648 a 2147483647. Ocupa 4 bytes.
- BIGINT:** Um inteiro grande. De 0 a 18446744073709551615 ou -9223372036854775808 a 9223372036854775807. Ocupa 8 bytes.
- DECIMAL:** Um número decimal. Para armazenar um valor monetário, por exemplo, você pode definir o tamanho dele como 8,2. Isso quer dizer que ele poderá ter 8 dígitos ao todo dos quais 2 são as casas decimais.
- DATE:** Data no formato 'AAAA-MM-DD'. Entre '1000-01-01' e '9999-12-31'.
- DATETIME:** Combinação de data e hora no formato 'AAAA-MM-DD HH:MM:SS'. Entre '1000-01-01 00:00:00' e '9999-12-31 23:59:59'.
- TIME:** Hora no formato 'HH:MM:SS'. A faixa é entre '-838:59:59' e '838:59:59'.
- YEAR:** Ano com 2 ou 4 dígitos (padrão 4 dígitos).

Campos String (texto)

- VARCHAR:** String com tamanho entre 1 e 255 caracteres.
- CHAR:** String com tamanho entre 1 e 255 caracteres. Para campos com tamanho fixo (CPF, CEP etc.) é mais vantagem usar o CHAR, porque ele ocupa um byte a menos que o VARCHAR. Por exemplo: você vai cadastrar o valor "abode" em um campo CHAR(5), ele vai ocupar 5 bytes, se você cadastrar em um VARCHAR(5), ele vai ocupar 6 bytes, porém se você cadastrar o valor "ab" em um campo CHAR(5) ele vai ocupar os mesmos 5 bytes, e o VARCHAR(5) vai ocupar 3 bytes (1 byte a mais do que a quantidade de caracteres).
- TINYTEXT** ou **TINYBLOB:** String com até 255 bytes.
- TEXT** ou **BLOB:** String com até 65535 bytes ou 64KB.
- MEDIUMTEXT** ou **MEDIUMBLOB:** String com até 16777215 bytes ou 16MB.
- LONGBLOB:** String com até 4294967295 bytes ou (4GB).

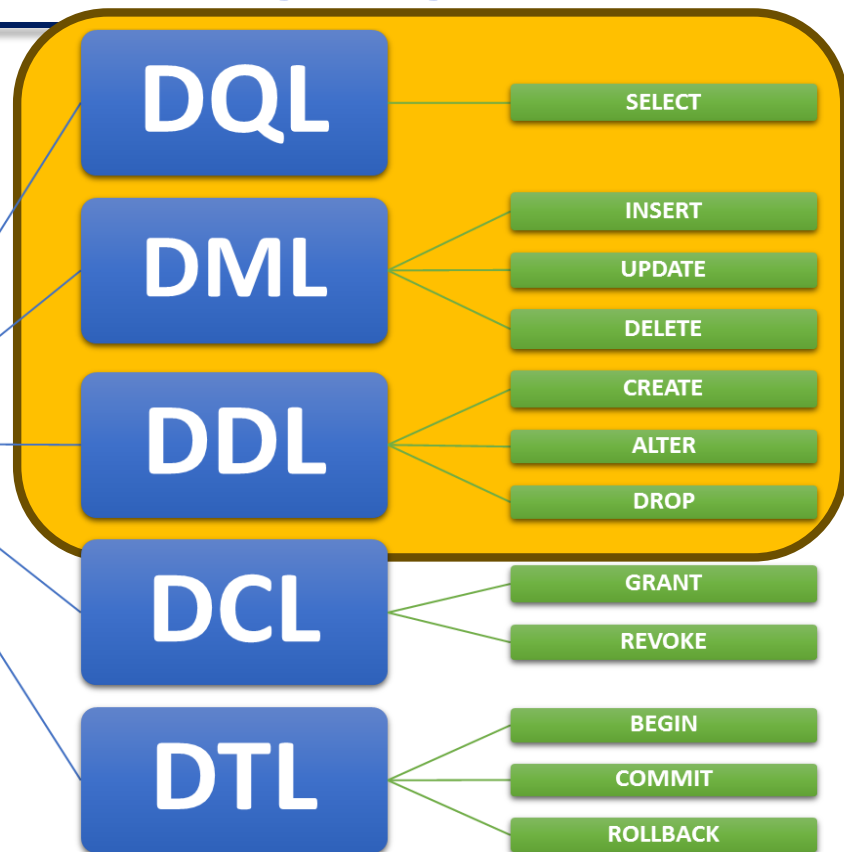


SQL – Structured Query Language

É uma linguagem estruturada que permite **criar, manipular e consultar** as informações armazenadas em um bando de dados

SQL

Padrão ANSI23



OBS.: O padrão SQL:2023 (ou ISO/IEC 9075:2023) é a versão mais recente do padrão da linguagem de consulta estruturada (SQL), adotada formalmente em junho de 2023.



Banco de Dados Relacional

São **coleções organizadas de dados** que se relacionam de forma a criar **algum sentido** e dar mais eficiência durante uma pesquisa.

Relacionamento

É a interconexão entre tabelas

Banco de Dados

Empregado

NumEmp	NomeEmp	Salário	Dept
032	J Silva	380	21
074	M Reis	400	25
089	C Melo	520	28
092	R Silva	480	25
112	R Pinto	390	21
121	V Simão	905	28
130	J Neves	640	28

Lota $\begin{matrix} \uparrow \\ N \\ 1 \end{matrix}$ Departamento

NumDept	NomeDept	Ramal
21	Pessoal	142
25	Financeiro	143
28	Técnico	144

Tabelas (Entidades)

Estruturas de dados organizadas em linhas e colunas

Linhas (Tuplas)

Coleções de colunas, também chamado de registros

Colunas

Campos que armazenam dados

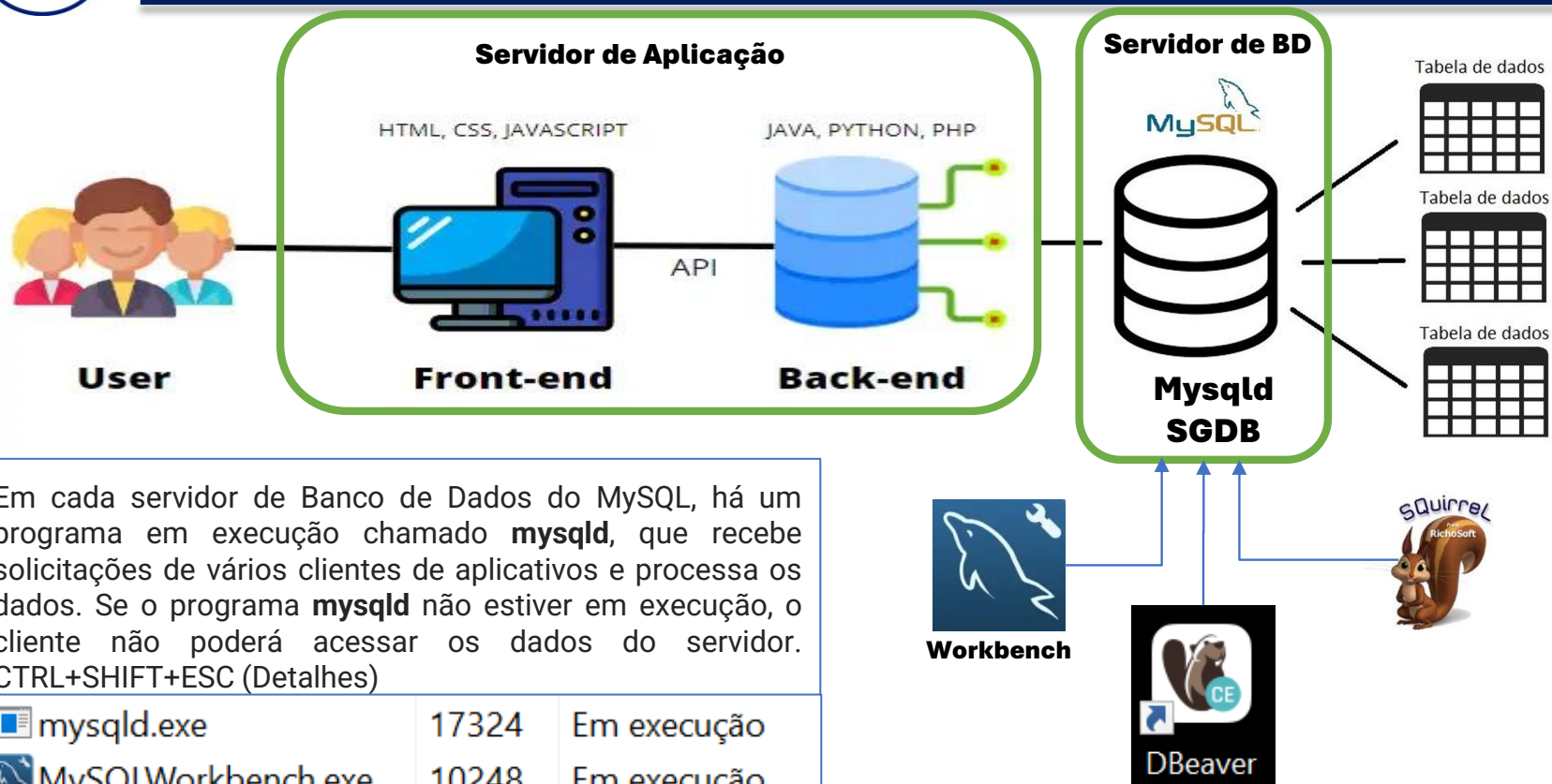


TREINA RECIFE
DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

Ferramentas necessárias ao nosso curso
MYSQL SERVER e MSQWORKBENCH



Servidor do MYSQL - SGDB





MYSQL Server e o Workbench

Google

download mysql community

Modo IA Tudo Videos Imagens Shopping Videos curtos Liv

MySQL
https://dev.mysql.com > downloads · Traduzir esta página

MySQL Community Downloads

MySQL Community Downloads · MySQL Yum Repository · MySQL APT Repository · MySQL Community Server · MySQL ND

MySQL Installer 8.0.44
MySQL Installer 8.0.44 ; (mysql-installer-web-community-8.0.44 ...

MySQL Community Server 8.4
MySQL Community Server 8.4.7 LTS ... We suggest that you use ...

MySQL Workbench 8.0.44
You do not need to download them if you use MySQL ... Download ...

MySQL Shell 9.5.0 Innovation
MySQL Community Downloads. MySQL Shell. General ...

Download - (mysql-9.5.0 ...
MySQL Community Server 9.5.0 Innovation ; (mysql-9.5.0 ...

Mais resultados de mysql.com »

MySQL Community Downloads

- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL NDB Cluster
- MySQL Router
- MySQL Shell
- MySQL Operator
- MySQL NDB Operator
- MySQL Workbench
- MySQL Installer for Windows

MySQL Community Downloads

MySQL Installer

General Availability (GA) Releases

Archives



MySQL Installer 8.0.44



Note: MySQL 8.0 is the final series with MySQL Installer. As of MySQL 8.1, use a MySQL product's MSI or Zip archive for installation. MySQL Server 8.1 and higher also bundle MySQL Configurator, a tool that helps configure MySQL Server.

Select Version:

8.0.44

Select Operating System:

Microsoft Windows

Windows (x86, 32-bit), MSI Installer

8.0.44

2.1 M

Download

(mysql-installer-web-community-8.0.44.0.msi)

MD5: F48ab9b8c2db55ee39ddF534d458167 Signature

Windows (x86, 32-bit), MSI Installer

8.0.44

558.3M

Download

(mysql-installer-community-8.0.44.0.msi)

MD5: 338dce4ac543dfc280664c857d265e3e | Signature



We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

<https://dev.mysql.com/downloads/installer/>
Siga o how to encaminhado pelo Whatsapp

Conhecendo o MySQL Workbench

The screenshot shows the MySQL Workbench interface with several annotations in blue boxes and arrows pointing to specific features:

- Executa todas as linhas**: Points to the 'Execute All' button (lightning bolt icon) in the Query Editor toolbar.
- Construção dos Scripts / Comandos**: Points to the Query Editor area where SQL scripts are written.
- Executa a linha que estiver o cursor**: Points to the 'Execute Current Statement' button (play icon) in the Query Editor toolbar.
- Área onde aparecerão o resultado da execução dos Scripts / comandos**: Points to the 'Result Grid' button in the right sidebar.
- Onde aparecerão o resultado das execuções (êxitos ou erros)**: Points to the 'Output' tab at the bottom of the interface.

Other visible elements include the 'No Context Help' message, the 'Ajuda baseada no contexto' (Context-based help) text, and the 'Schema selecionado' (Selected schema) message.

Output

#	Time	Action	Message	Duration / Fetch
✓ 40	17:05:58	select * from teste LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
✓ 41	17:06:58	EXPLAIN select * from teste	OK	0.000 sec



Conhecendo Workbench

- **Charset (Character Set):** Define o conjunto de caracteres que pode ser usado para armazenar dados de texto na sua base de dados. Por exemplo, o utf8mb4 é um conjunto de caracteres abrangente que suporta todos os caracteres Unicode, incluindo acentos, símbolos e emojis, sendo o mais recomendado atualmente.
- **Collation:** Define as regras para comparar e ordenar os caracteres do conjunto definido pelo charset.
- **Engine:** Define o "motor" de armazenamento que a tabela usará para gerenciar os dados. Cada motor possui características de desempenho e funcionalidades diferentes.
 - **InnoDB:** É o motor de armazenamento mais comum e recomendado para a maioria dos casos. Ele oferece alta confiabilidade, chaves estrangeiras e recuperação de falhas.

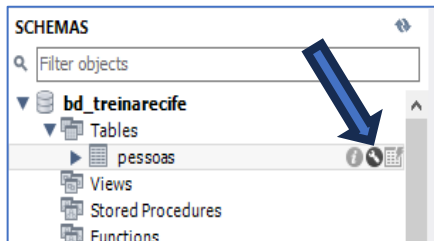
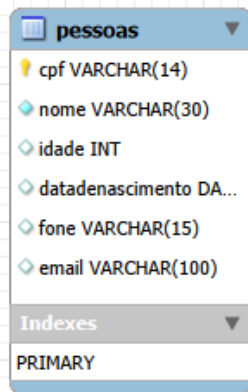


Table Name: Schema:

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
cpf	VARCHAR(14)	☒	☒	☐	☐	☐	☐	☐	☐	
nome	VARCHAR(30)	☐	☒	☐	☐	☐	☐	☐	☐	
idade	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
datadenascimento	DATE	☐	☐	☐	☐	☐	☐	☐	☐	NULL
fone	VARCHAR(15)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
email	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

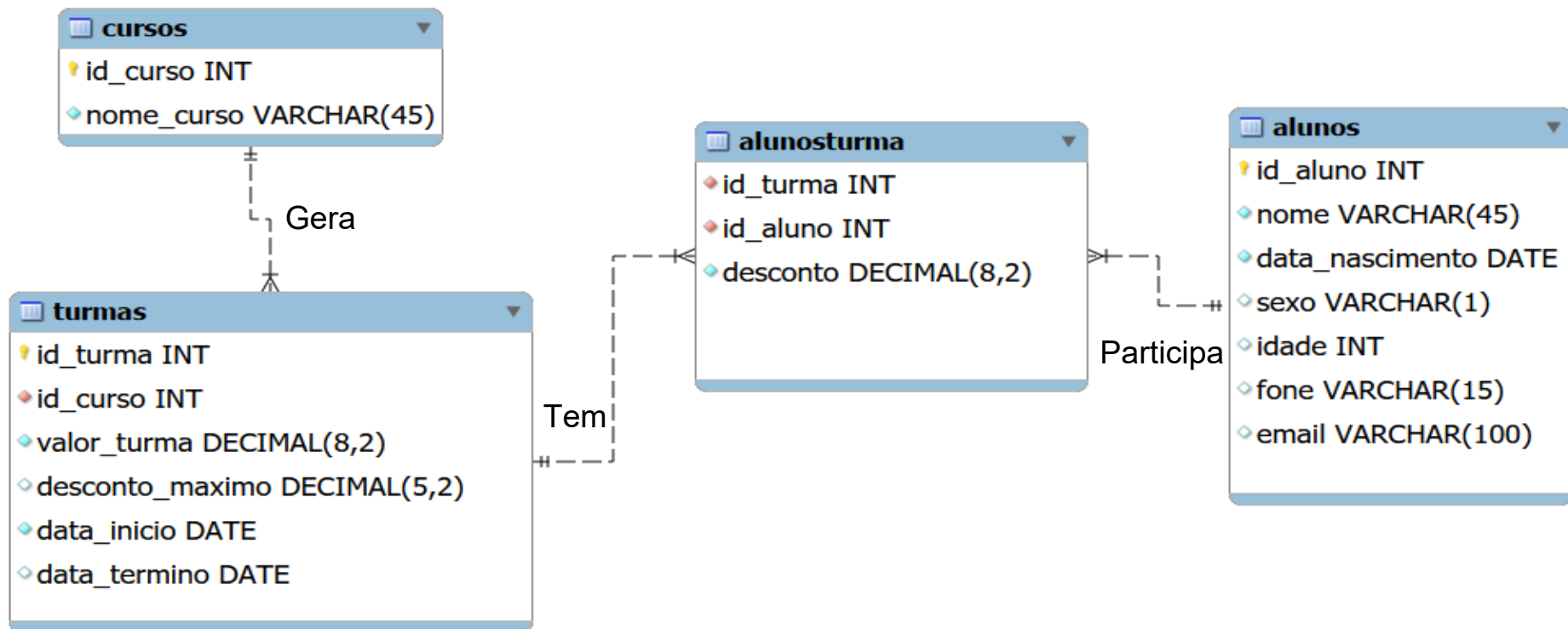


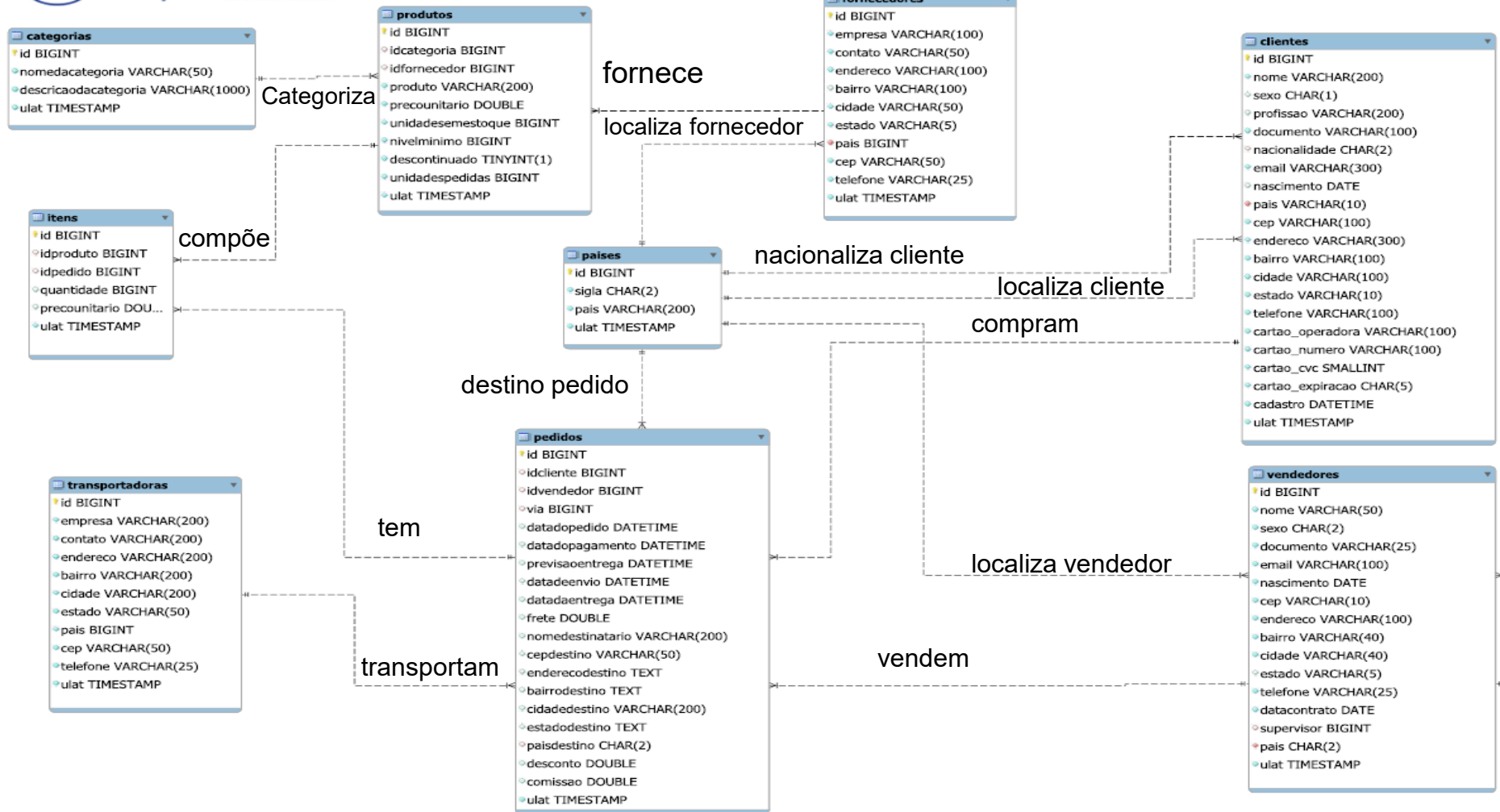
Sigla	Significado	Descrição
PK	Primary Key (Chave Primária)	Indica que a coluna é a chave primária da tabela. O valor nesta coluna deve ser único para cada registro e não pode ser nulo.
NN	Not Null (Não Nulo)	Garante que a coluna não pode ter um valor nulo. O preenchimento do campo é obrigatório ao inserir um novo registro.
UQ	Unique (Único)	Força que todos os valores na coluna sejam únicos, mas, ao contrário da chave primária, pode permitir valores nulos.
B	Binary (Binário)	Trata os dados em colunas de texto (como VARCHAR ou CHAR) como dados binários, onde a comparação e a ordenação são baseadas nos valores numéricos dos bytes.
UN	Unsigned (Sem Sinal)	Aplicável a tipos de dados numéricos. Indica que a coluna não pode conter valores negativos, o que duplica o limite superior do intervalo de valores possíveis (por exemplo, INT vai de 0 até 4294967295).
ZF	Zero Fill (Preencher com Zero)	Usado com tipos de dados numéricos e a opção UN . Preenche o valor com zeros à esquerda até o comprimento definido da coluna. Por exemplo, se uma coluna INT(4) com ZF recebe o valor 21, ele será armazenado como 0021.
AI	Auto Increment (Auto Incremento)	Automaticamente gera um número único e sequencial para a coluna a cada novo registro. É comumente usado em chaves primárias numéricas.
G	Generated (Gerada)	Indica que a coluna tem um valor calculado com base em uma expressão ou em outras colunas. O valor não é inserido manualmente, mas sim gerado pelo banco de dados.



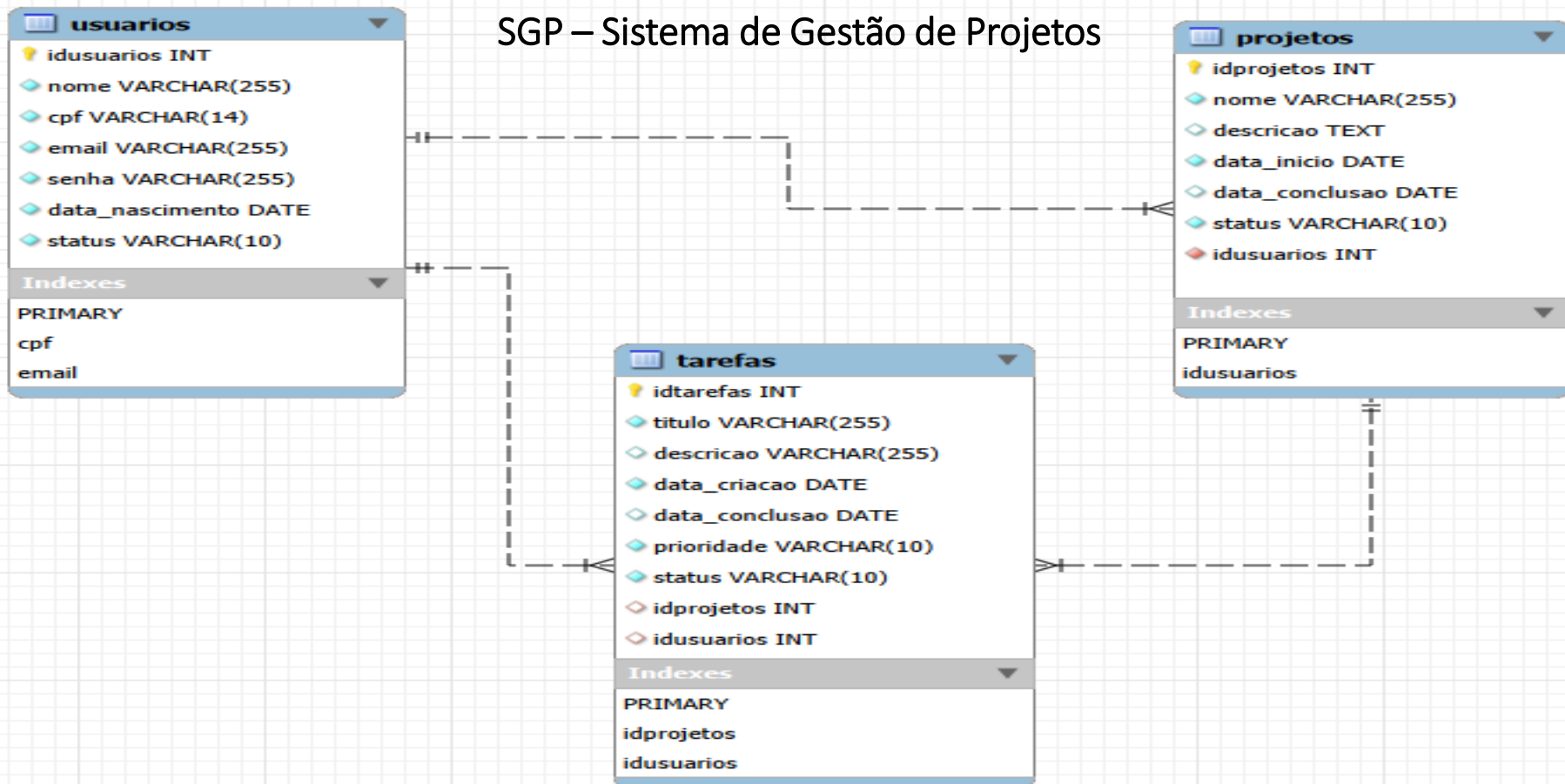
Entendendo Modelos de Dados

Curso_Treinarecife





SGP – Sistema de Gestão de Projetos





TREINA RECIFE
DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

DDL – Data Definition Language
CREATE , ALTER e DROP
Linguagem de Definição de Dados



Criando e Eliminando um banco no MySQL

Criando Banco de Dados

```
CREATE DATABASE [IF NOT EXISTS] nome_bd;
```

Disponibilizando para uso

```
USE nome_bd;
```

Eliminando um Banco de Dados

```
DROP DATABASE [IF EXISTS] nome_bd;
```

Tudo entre [] é opcional. A vírgula separa definições internas.



#dica: O Workbench chama o Banco de Dados de **SCHEMA**

1

Criação

CREATE DATABASE cria um novo SCHEMA no banco de dados.

2

Proteção

IF NOT EXISTS evita erro quando o SCHEMA já existe., deve ser usado sem os []

Exemplo:

-- Criando o Banco de Dados

```
CREATE DATABASE IF NOT EXISTS  
    bd_treinarecife;
```

-- Setando o Banco de Dados para uso

```
USE bd_treinarecife
```

-- Eliminando o meu Banco de Dados

```
DROP DATABASE IF EXISTS bd_treinarecife;
```



Tipo de Dados mais comuns do MySQL

Tipo	Strings – Alfanumérico
CHAR(tam)	String que ocupa tamanho fixo entre 0 e 255 caracteres
VARCHAR(tam)	String de tamanho variável, até 65535 caracteres
Tipo	Númérico – Inteiro / Decimal
INT	SIGNED: -2,147.483.648 e 2.147.483.647 UNSIGNED: 0 a 4.294.967.295
DECIMAL(m,d)	Ponto decimal com "m" dígitos no total (precisão) e "d" casas decimais (escala); o padrão é 10,0; "m" vai até 65 e "d" até 30.
Tipo	Data / Hora
DATE	Uma data de 01/01/1000 a 31/12/9999, no formato YYYY-MM-DD
DATETIME	Uma combinação de data e hora de 01/01/1000 00:00:00 a 31/12/9999 23:59:59, no formato YYYY-MM-DD HH:MM:SS
TIME	Hora apenas, no formato HH:MM:SS
Tipo	Lógico
BOOLEAN	BOOLEAN – Valores binários 0(FALSE) / 1(TRUE); Na verdade, é um apelido para o tipo TINYINT(1) – Pequeno Inteiro(Minusculo)



#dica: Usamos CHAR quando as colunas tem o mesmo tamanho. Ex: UF, SIGLA



Criando uma Tabela no Banco de Dados

```
CREATE TABLE [IF NOT EXISTS] nome_tabela (  
  nome_coluna1 tipo  
    [SIGNED | UNSIGNED]  
    [NOT NULL | NULL]  
    [DEFAULT valor]  
    [AUTO_INCREMENT]  
    [COMMENT 'texto'],  
  
  nome_colunaN tipo  
    [SIGNED | UNSIGNED]  
    [NOT NULL | NULL]  
    [DEFAULT valor]  
    [COMMENT 'texto'],  
  
  PRIMARY KEY (coluna1, coluna2, ...),  
  FOREIGN KEY (coluna)  
    REFERENCES tabela_pai (coluna_pai)  
);
```

Tudo entre [] é opcional. A vírgula separa definições internas.

Resumo: CREATE TABLE define a “forma oficial” dos dados antes de inserir registros.

1

Criação

CREATE TABLE cria uma nova tabela no banco de dados.

2

Proteção

IF NOT EXISTS evita erro quando a tabela já existe., deve ser usado sem os []

3

Colunas

Cada coluna recebe nome, tipo e regras de preenchimento.

4

Identidade

PRIMARY KEY identifica de forma única de cada registro.

5

Relação

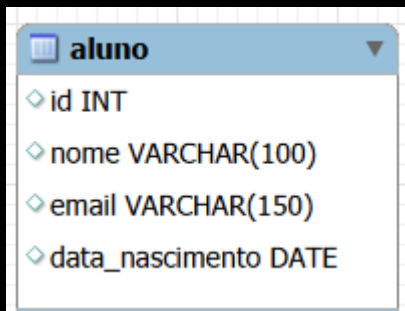
FOREIGN KEY liga uma coluna à chave de uma tabela pai.

Default: **unsigned** e **null**



Ex 1: criando uma tabela simples

```
CREATE TABLE aluno (  
  id INT,  
  nome VARCHAR(100),  
  email VARCHAR(150),  
  data_nascimento DATE  
);
```



Entendendo o exemplo

1

aluno

Nome da nova tabela.

2

id INT

Número inteiro para identificar o aluno.

3

nome

VARCHAR armazena texto com tamanho máximo 100.

4

email

VARCHAR armazena texto com tamanho máximo 150

5

data_nascimento

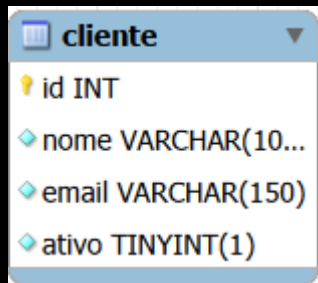
DATE armazena uma data.



Ex 2: usando NOT NULL, DEFAULT e AUTO_INCREMENT

Adicionando regras para controlar como cada coluna pode ser preenchida.

```
CREATE TABLE cliente (  
  id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  PRIMARY KEY (id)  
);
```



Entendendo o exemplo

1

UNSIGNED

Impede valores negativos no campo numérico.

2

NOT NULL

O preenchimento da coluna passa a ser obrigatório.

3

DEFAULT TRUE

Usa TRUE quando nenhum valor for informado.

4

AUTO_INCREMENT

O banco gera o próximo id automaticamente.

5

PRIMARY KEY

Define id como identificador único.



Ex 3: chave primária composta

Uma chave primária também pode ser formada pela combinação de duas ou mais colunas.

```
CREATE TABLE matricula (  
  aluno_id INT NOT NULL,  
  turma_id INT NOT NULL,  
  data_matricula DATE NOT NULL,  
  PRIMARY KEY (aluno_id, turma_id)  
);
```

matricula	
aluno_id	INT
turma_id	INT
data_matricula	DATE

Entendendo o exemplo

1

matricula

Representa a inscrição de um aluno em uma turma.

2

aluno_id

Identifica o aluno participante.

3

turma_id

Identifica a turma escolhida.

4

PRIMARY KEY

A combinação aluno + turma não pode se repetir.

5

Resultado

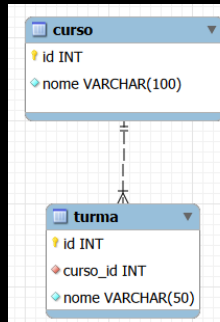
O mesmo aluno pode estar em turmas diferentes.



Ex 4: relacionando duas tabelas com FOREIGN KEY

A chave estrangeira cria uma relação entre registros de tabelas diferentes.

```
CREATE TABLE curso (  
  id INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(100) NOT NULL,  
  PRIMARY KEY (id)  
);  
  
CREATE TABLE turma (  
  id INT NOT NULL AUTO_INCREMENT,  
  curso_id INT NOT NULL,  
  nome VARCHAR(50) NOT NULL,  
  PRIMARY KEY (id),  
  FOREIGN KEY  
    (curso_id) REFERENCES curso(id)  
);
```



Entendendo o exemplo

1

curso.id

É a chave primária da tabela pai.

2

turma.curso_id

Guarda o código do curso relacionado.

3

FOREIGN KEY

Declara que curso_id é uma chave estrangeira.

4

REFERENCES

Indica qual tabela e coluna serão referenciadas.

5

Integridade

Uma turma só referencia um curso existente.



Como pensar antes de escrever o **CREATE TABLE**

Use esta sequência mental para transformar um requisito de negócio em estrutura SQL.

1 Qual tabela?

Defina a entidade: aluno, curso, produto, pedido...

2 Quais dados?

Liste as colunas necessárias para representar a entidade.

3 Qual tipo?

Escolha INT, VARCHAR, DATE, DECIMAL, BOOLEAN etc.

4 Quais regras?

Decida NOT NULL, DEFAULT, UNSIGNED, AUTO_INCREMENT...

5 Qual identidade?

Defina a PRIMARY KEY.

6 Quais relações?

Crie FOREIGN KEY quando houver tabelas relacionadas.

CREATE TABLE fica mais simples quando você separa o problema em decisões pequenas e sequenciais.



Eliminando uma Tabela

Eliminando uma tabela

```
DROP TABLE [IF EXISTS] nome_bd;
```

Tudo entre [] é opcional. A vírgula separa definições internas.

Exemplo:

```
DROP TABLE IF EXISTS aluno;
```



Vamos Exercitar

- ❑ **Ex01** - Construa um script que crie um banco de dados chamado **bd_treinarecife**. Nele crie uma tabela denominada **pessoas** com a estrutura abaixo, não se preocupe, neste momento, em popular a tabela.

The screenshot shows a database schema editor window titled 'pessoas'. It contains a list of fields with their data types and constraints:

- cpf VARCHAR(14) (Primary Key)
- nome VARCHAR(30)
- idade INT
- datadenascimento DATE
- fone VARCHAR(15)
- email VARCHAR(100)

Below the fields, there is a section for 'Indexes' with a dropdown menu showing 'PRIMARY'.



Relacionando Tabelas – Foreign key

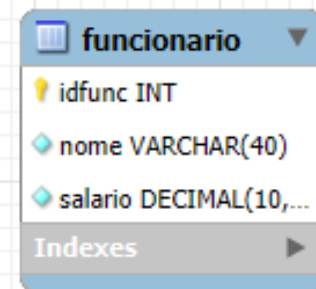
Tabela PAI

```
CREATE TABLE IF NOT EXISTS funcionario (  
  idfunc int unsigned not null ,  
  nome varchar(40) not null,  
  salario decimal (10,2) not null,  
  primary key (idfunc)  
);
```

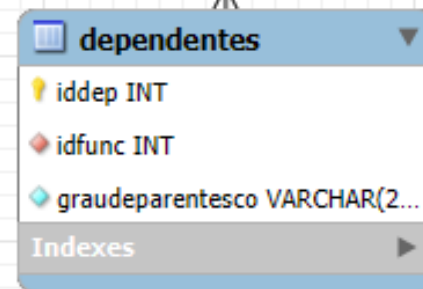
Tabela FILHA

```
CREATE TABLE IF NOT EXISTS dependentes (  
  iddep int unsigned not null,  
  idfunc int unsigned not null,  
  graudeparentesco varchar(20) not null,  
  primary key (iddep),  
  foreign key (idfunc) references funcionario (idfunc)  
);
```

PAI



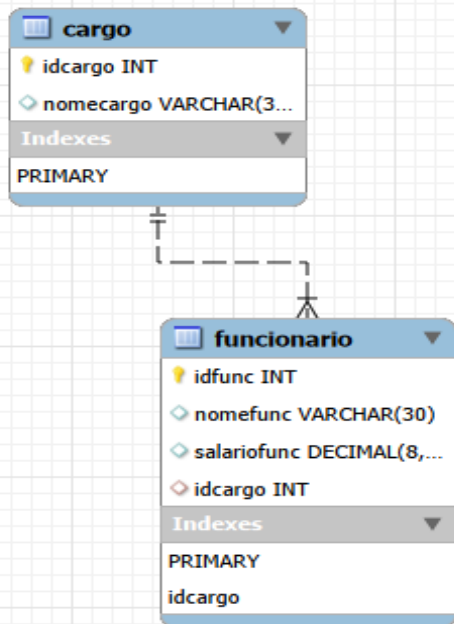
FILHA





Vamos Exercitar

- ❑ **Ex02** - Construa um script que acrescente no banco **bd_treinarecife** tabelas **cargo** e **funcionario** conforme layout abaixo:





Adicionando colunas na Tabela



#dica: obrigatório informar o nome da coluna e o tipo. Demais características são opcionais

```
ALTER TABLE table_name
  ADD column_name1 datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'],
  ....
  ADD column_nameN datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'];
```

Tudo entre [] é opcional. A vírgula separa definições internas.

Exemplo:

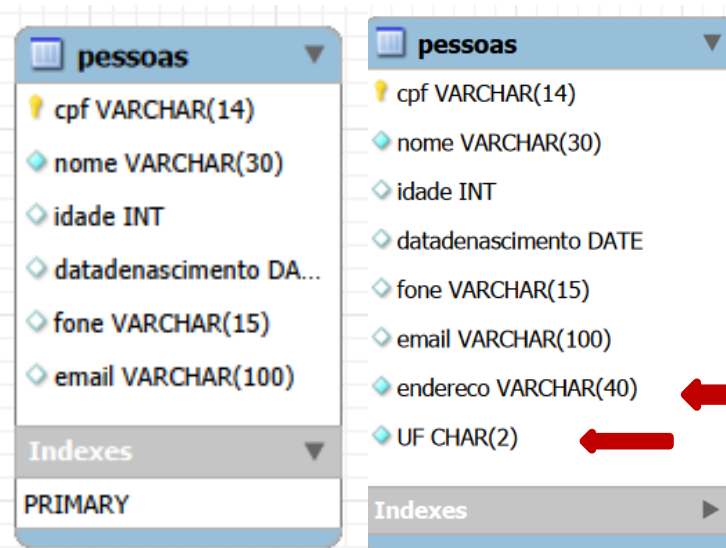
Adicionando as colunas endereço e UF na tabela Pessoas

USE bd_treinarecife;

ALTER TABLE pessoas

ADD endereco varchar(40) not null comment 'Endereço',

ADD UF char(2) not null default 'PE' comment 'Estado';





Substituindo e Modificando colunas na Tabela

```
ALTER TABLE table_name
CHANGE column_name1 new_column_name1 datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'],
.....
CHANGE column_nameN new_column_nameN datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'];
```

Tudo entre [] é opcional. A vírgula separa definições internas.

```
ALTER TABLE table_name
MODIFY column_name1 datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'],
.....
MODIFY column_nameN datatype
    [signed|unsigned]
    [not null|null]
    [default valor]
    [comment 'texto'];
```

Tudo entre [] é opcional. A vírgula separa definições internas.



Exemplo

Substituindo as colunas da tabela **funcionario**:
nomefunc por **nome varchar(20)** e
salariofunc por **salario decimal(6,2)**

Modificando a coluna **nomecargo** para **varchar(40)**
na tabela **cargo**.

USE bd_treinarecife;

ALTER TABLE funcionario
CHANGE nomefunc nome varchar(20),
CHANGE salariofunc salario decimal(6,2);

ALTER TABLE cargo
MODIFY nomecargo varchar(40);

cargo	
idcargo	INT
nomecargo	VARCHAR(30)
Indexes	

funcionario	
idfunc	INT
nomefunc	VARCHAR(30)
salariofunc	DECIMAL(8,2)
idcargo	INT
Indexes	
PRIMARY	
idcargo	

cargo	
idcargo	INT
nomecargo	VARCHAR(40)
Indexes	

funcionario	
idfunc	INT
nome	VARCHAR(20)
salario	DECIMAL(6,2)
idcargo	INT
Indexes	
PRIMARY	
idcargo	





Removendo colunas de uma Tabela

```
ALTER TABLE table_name  
DROP column_name1,  
....  
DROP column_nameN;
```

Ex: Eliminando as colunas **endereço** e **UF** da tabela **pessoas**

USE bd_treinarecife;

ALTER TABLE pessoas
DROP column endereco,
DROP column UF;

pessoas	
cpf	VARCHAR(14)
nome	VARCHAR(30)
idade	INT
datadenascimento	DATE
fone	VARCHAR(15)
email	VARCHAR(100)
endereco	VARCHAR(40)
UF	CHAR(2)
Indexes	

pessoas	
cpf	VARCHAR(14)
nome	VARCHAR(30)
idade	INT
datadenascimento	DA...
fone	VARCHAR(15)
email	VARCHAR(100)
Indexes	
PRIMARY	



Utilizando as três forma em um comando

USE bd_treinarecife;

ALTER TABLE cliente

CHANGE nome nomecliente varchar (40),

MODIFY email varchar(100),

DROP ativo;

Antes

cliente
id INT
nome VARCHAR(100)
email VARCHAR(150)
ativo TINYINT(1)
Indexes

Depois

cliente
id INT
nomecliente VARCHAR(40)
email VARCHAR(100)
Indexes



Resumo

Adicionando Colunas

```
ALTER TABLE table_name  
ADD column_name1 datatype  
[signed|unsigned][not null|null][default valor][comment 'texto'];
```

Substituindo ou Modificando Colunas

```
ALTER TABLE table_name  
CHANGE column_name new_column_name1 datatype  
[signed|unsigned] [not null|null] [default valor] [comment 'texto'] ;  
ou  
ALTER TABLE  
MODIFY column_name datatype  
[signed|unsigned] [not null|null] [default valor] [comment 'texto'] ;
```

Eliminando colunas:

```
ALTER TABLE table_name  
DROP column_name,  
DROP column_name;
```



Vamos Exercitar

❑ **Ex03** – No Banco **bd_treinarecife**:

a) Crie a tabela **aluno** com a estrutura ao lado todas as colunas **not null** e a **matricula** como **PK**.

b) Adicione na tabela **aluno** os campos:

- **idade** int not null;
- **endereco** varchar(45) not null
- **UF** char(2) not null com default 'PE'

c) Altere a coluna **nome** para **nomealuno varchar(45)**

d) Remova as colunas **idade** e **endereco**.

aluno	
matricula	INT
nomealuno	VARCHAR(45)
UF	CHAR(2)
Indexes	

aluno	
matricula	INT
nome	VARCHAR(30)
Indexes	

aluno	
matricula	INT
nome	VARCHAR(30)
idade	INT
endereco	VARCHAR(45)
UF	CHAR(2)
Indexes	

aluno	
matricula	INT
nomealuno	VARCHAR(45)
UF	CHAR(2)
Indexes	



TREINA RECIFE

DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

DML – Data Manipulation Language

INSERT , UPDATE e DELETE

Linguagem de Manipulação de dados



INSERT – Inserindo Linhas

Inserindo Linhas em uma Tabela

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

Exemplo:

```
create table exinsert (c1 int, c2 int, c3 varchar (15));
```

```
-- Especificando as colunas e invertendo a ordem
```

```
insert into exinsert (c1, c3, c2) values (11 , "Rogerio", 33);
```

```
-- Sem especificar as colunas - obrigado seguir a ordem
```

```
insert into exinsert values (22 , 44, 'Qualquer coisa');
```

```
-- Caso a coluna aceite NULL não precisa ser informado
```

```
insert into exinsert (c1, c3) values (99, 'Coluna C2 Nulo');
```

#dica: não é necessário seguir a mesma ordem das colunas na instrução **CREATE TABLE**, quando usamos a descrição das colunas, pois a associação é posicional. Caso os nomes sejam omitidos a ordem deve ser respeitada. Caso a coluna permita NULL não precisam ser fornecidas, pois receberão valor NULL ou valor padrão definido.



exinsert

◇ c1 INT

◇ c2 INT

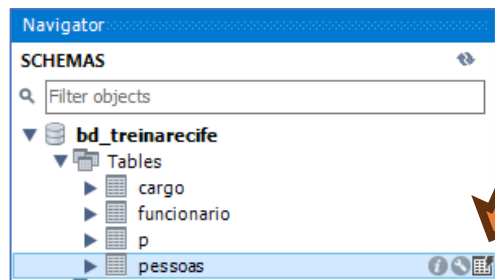
◇ c3 VARCHAR(15)



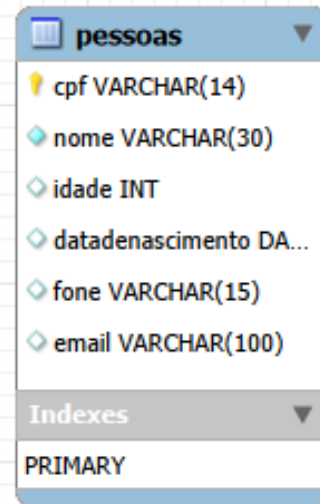
INSERT – Inserindo Linhas na tabela pessoas

Exemplo inserindo múltiplas linhas:

```
INSERT INTO pessoas (cpf, nome, idade, datadenascimento, fone, email) VALUES  
( '111.222.333-44', 'Rogério Aguiar', 40, '2000-12-14', '9-45611239', 'rogerio@email.com'),  
( '222.333.444-55', 'Maria da Luz', 18, '2000-12-15', '9-12345678', 'luz@yahoo.com'),  
( '333.444.555-66', 'Fátima Bernardes', 19, '2000-12-16', '9-36985214', 'fatima@email.com'),  
( '444.555.666-77', 'Carlos Alencar', 45, '2000-12-17', '9-32145698', 'alencar@gmail.com')`
```



Result Grid						
Filter Rows:						
Edit: Export/Import: Wrap Cell Content:						
	cpf	nome	idade	datadenascimento	fone	email
▶	111.222.333-44	Rogério Aguiar	40	2000-12-14	9-45611239	rogerio@email.com
	222.333.444-55	Maria da Luz	18	2000-12-15	9-12345678	diniz@yahoo.com
	333.444.555-66	Fátima Bernardes	19	2000-12-16	9-36985214	helio@email.com
	444.555.666-77	Carlos Alencar	45	2000-12-17	9-32145698	claudia@gmail.com
	NULL	NULL	NULL	NULL	NULL	NULL





Vamos Exercitar

EX04: Inserir na tabela **peessoas** as seguintes linhas

Cpf	Nome	Idade	Dt.Nasc.	Fone	email
666.666.666-66	Maria das Dores	40	14/09/1984	9-66666666	maria@email.com
888.888.888-88	Fátima Alencar	20	15/12/2004	9-88888888	fatima@yahoo.com

peessoas
cpf VARCHAR(14)
nome VARCHAR(30)
idade INT
datadenascimento DA...
fone VARCHAR(15)
email VARCHAR(100)
Indexes
PRIMARY

```
INSERT INTO pessoas (cpf, nome, idade, datadenascimento, fone, email) VALUES  
( '666.666.666-66', 'Maria das Dores', 40, '1984-09-14', '9.66666666', 'maria@email.com'),  
( '888.888.888-88', 'Fátima Alencar' , 20, '2004-12-15', '9.88888888', 'fatima@yahoo.com');
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	cpf	nome	idade	datadenascimento	fone	email
▶	666.666.666-66	Maria das Dores	40	1984-09-14	9.66666666	maria@email.com
	888.888.888-88	Fátima Alencar	20	2004-12-15	9.88888888	fatima@yahoo.com
✱	NULL	NULL	NULL	NULL	NULL	NULL



UPDATE – Atualizando Linhas

Alterando Linhas em uma Tabela

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
[WHERE condition];
```



Ex:

```
UPDATE pessoas  
SET nome = 'Rogério Aguiar';
```

Atenção: Sem
o **where**
Todas as
linha são
afetadas

Antes

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Maria das Dores	40	1984-09-14	9-88888888	maria@email.com
888.888.888-88	Fátima Alencar	20	2004-12-15	9-88888888	fatima@gmail.com

Depois

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Rogério Aguiar	40	1984-09-14	9-88888888	maria@email.com
888.888.888-88	Rogério Aguiar	20	2004-12-15	9-88888888	fatima@gmail.com

#: o Workbench vem com a flag **SAFE UPDATE ON**, isso garante que você não consiga fazer **UPDADE** ou **DELETE** sem a indicação do registro a ser alterado. Para alterar esta opção vá para **Edit | Preferences | SQL Editor**, rolar até o final e desmarcar a opção **Safes Updates(rejects UPDATES and DELETES with no restrictions.**

Após marcar ou desmarcar precisa reconectar o banco Menu **Query > Reconnect to Server**

Ou

```
SET SQL_SAFE_UPDATES = 0 (OFF)  
SET SQL_SAFE_UPDATES = 1 (ON)
```

Obs.: MUITO CUIDADO AO USAR



UPDATE – Atualizando Linhas

Alterando Linhas em uma Tabela usando a clausula **WHERE**

Ex:

UPDATE pessoas **SET** nome = 'Maria das Dores' **WHERE** cpf = '666.666.666-66';

UPDATE pessoas **SET** nome = 'Fátima Alencar' **WHERE** cpf = '888.888.888-88';



Atenção: com o **where** apenas a(s) linha(s) que atendam a condição especificada, são alterada(s).

Antes

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Rogério Aguiar	40	1984-09-14	9-88888888	maria@email.com
888.888.888-88	Rogério Aguiar	20	2004-12-15	9-88888888	fatima@gmail.com

Depois

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Maria das Dores	40	1984-09-14	9-88888888	maria@email.com
888.888.888-88	Fátima Alencar	20	2004-12-15	9-88888888	fatima@gmail.com



DELETE – Eliminando Registros

Eliminando Registros em uma Tabela

DELETE FROM *table_name* [**WHERE** *condition*];



Ex: **DELETE FROM** pessoas;

Antes

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Maria das Dores	40	1984-09-14	9-888888888	maria@email.com
777.777.777-77	João Olímpio	18	2006-08-01	9-77777777	joao@yahoo.com
888.888.888-88	Fátima Alencar	20	2004-12-15	9-888888888	fatima@gmail.com

Depois

cpf	nome	idade	datanascimento	fone	email
NULL	NULL	NULL	NULL	NULL	NULL

Ex: **DELETE FROM** pessoas **WHERE** cpf = '777.777.777-77';

cpf	nome	idade	datanascimento	fone	email
666.666.666-66	Maria das Dores	40	1984-09-14	9-888888888	maria@email.com
888.888.888-88	Fátima Alencar	20	2004-12-15	9-888888888	fatima@gmail.com

Eliminando todos os registros em uma Tabela

TRUNCATE *table_name*;



#: o Workbench vem com a flag **SAFE UPDATE ON**, isso garante que você não consiga fazer **UPDADE** ou **DELETE** sem a indicação do registro a ser alterado.



#:**Dica**: este comando independe da flag **SAFE UPDATE ON**, portanto muito cuidado ao usá-lo.



TREINA RECIFE
DESENVOLVIMENTO GERENCIAL E PROFISSIONAL

DQL – Data Query Language

SELECT

Linguagem de Consulta de Dados



SELECT .. FROM.. WHERE (Condição)....

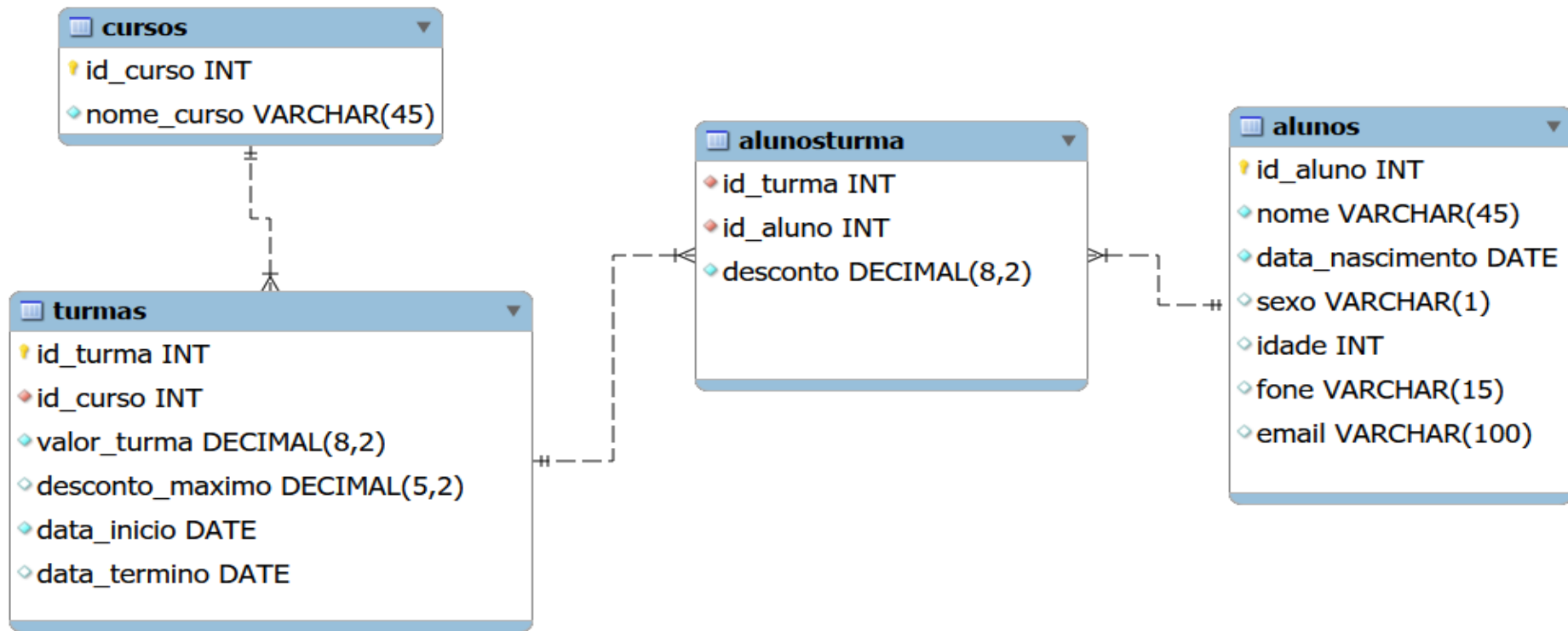
```
SELECT column1, column2, ...  
FROM table_name  
[WHERE condition1 AND/OR condition2]  
[GROUP BY column name(s)]  
[HAVING condition]  
[ORDER BY column1, column2, ... ASC|DESC]  
[LIMIT number];
```



#Observação: O MySQL não diferencia letras acentuadas **por padrão** devido à configuração de **collation** (ordenamento e comparação de caracteres) da tabela ou do banco de dados, ele utiliza uma **collation** padrão chamada **utf8mb4_general_ci** que é **insensível a acentos e maiúsculas/minúsculas**. Onde o sufixo **_ci** significa "**case-insensitive**" (insensível a maiúsculas/minúsculas) e também inclui a indiferença a caracteres acentuados.

Utilize **SHOW VARIABLES LIKE 'collation_server';** para saber a collation que seu mysql está utilizando.

Curso_Treinarecife





Importando o Banco para o Workbench

1. Acessar o **MySQL WorkBench** e iniciar a conexão com o Banco de Dados;
2. Ao acessar o Banco, vá na aba de "**Administration**" do lado esquerdo do programa e clique na opção "**Data Import/Restore**";
3. Em seguida, na tela que aparecer, faça os seguintes procedimentos:
 - a) Selecione o arquivo que contém o dump;
 - b) Marque a opção "*Import to Self-Contained File*";
 - c) Escolha o banco que quer restaurar;
4. Por fim, clique em "**Start Import**".

new connection
Data Import

Import from Disk | Import Progress

Import Options

☐ Import from Dump Project Folder C:\Users\Treina Recife\Documents\dumps ...

Select the Dump Project Folder to import. You can do a selective restore.

Load Folder Contents

☒ Import from Self-Contained File C:\Users\Treina Recife\Documents\dumps\Dump20241026 (4).sql ...

Select the SQL/dump file to import. Please note that the whole file will be imported.

Default Schema to be Imported To

Default Target Schema: curso_treinarecife

New... The default schema to import the dump into.
NOTE: this is only used if the dump file doesn't contain its schema, otherwise it is ignored.

Select Database Objects to Import (only available for Project Folders)

Imp...	Schema
--------	--------

Imp...	Schema Objects
--------	----------------

Dump Structure and Dat... Select Views Select Tables Unselect All

Import Completed

Start Import



Parametros utilizadas nas colunas do Select

Operador	Descrição	Exemplo	Efeito sobre
Concat	Concatenação	Select concat(nome,telefone,...)	Concatena colunas a outras colunas
AS	Alias - Renomeia um cabeçalho de coluna	Select nome as 'Nome da Pessoa'	Gera um apelido para a coluna, a palavra-chave AS é opcional
Char_length()	Comprimento do conteúdo de uma coluna	Select nome, char_length(nome)	Retorna o nome e seu comprimento consider
Distinct	Linhas distintas	Select distinct nome, telefone	Se houver linhas duplicadas traz apenas uma ocorrência dela.
Operações	Operações Aritméticas	Select cola+colB	



Clausula WHERE (Condição)

Operador	Descrição	Exemplo	Efeito sobre
=	Igual	WHERE cpf = 111;	Linhas cujo cpf for igual a 111
>	Maior que	WHERE cpf > 222;	Linhas cujo cpf for maior que 222
<	Menor que	WHERE cpf < 222;	Linhas cujo cpf for menor que 222
>=	Maior ou igual		
<=	Menor ou igual	WHERE cpf >= 222;	Linhas cujo cpf for maior ou igual que 222
<> ou !=	Diferente	WHERE cpf != 222;	Linhas cujo cpf for diferente de 222
BETWEEN	Entre uma faixa de valor	WHERE cpf BETWEEN 111 AND 333; cpf >= 111 and cpf <= 333;	Linhas entre cujo cpf esteja entre os valores 111 e 333, inclui os valores 111 e nem o 333
LIKE NOT LIKE	Pesquisa por parte,tanto para campos numéricos quanto textos. Combinação dos caracteres “_” e “%”	WHERE nome LIKE ‘c%’; WHERE nome LIKE ‘%a’; WHERE nome LIKE ‘%a%’; WHERE nome LIKE ‘_a%’; WHERE cpf LIKE ‘_1_’;	Linhas cujo nome inicie pela letra “C” ou “c” Linhas cujo nome termine com a letra “A,a,á,à” Linhas cujo nome contenha a letra “A,a,á,à” Linhas cujo nome possua a letra “a” na segunda posição Linhas cujo cpf possua três posições com a segunda = 1
IN	Multiplos valores	WHERE cpf IN (111,333);	Linhas cujo cpf seja igual a 111 ou 333;
NOT IN	Multiplos valores	WHERE cpf NOT IN (111,333);	Linhas cujo cpf seja diferente de 111 e 333
OR	Múltiplas condições	WHERE cpf = 111 OR cpf = 333	Linhas cujo cpf seja igual a 111 ou 333
AND	Múltiplas condições	WHERE cpf = 111 AND nome = "Rogério Aguiar"	Linhas cujo cpf seja igual a 111 e nome igual a “Rogério Aguiar”



Exemplo: *SELECT .. FROM.. WHERE (Condição)*

Ex:

```
SELECT * FROM alunos;
```

Retorna todas as
colunas e todas as linhas

```
SELECT nome, data_nascimento FROM alunos;
```

Retorna apenas as colunas
nome e **data_nascimento**
de todas as linhas

```
SELECT Idade, nome FROM alunos ORDER BY  
nome;
```

Retorna apenas as colunas **idade**
e **nome** de todas as linhas,
ordenadas pelo nome(A-Z)

```
SELECT * FROM alunos LIMIT 5;
```

Retorna todas as colunas das
primeira 5 linhas encontradas

```
SELECT nome , idade , sexo FROM alunos  
WHERE fone = '(81) 98424-2055'
```

Retorna o nome, idade sexo dos
alunos que tem o telefone igual
(81) 98424-2055

alunos	
id_aluno	INT
nome	VARCHAR(45)
data_nascimento	DATE
sexo	VARCHAR(1)
idade	INT
fone	VARCHAR(15)
email	VARCHAR(100)



Vamos Exercitar - SELECT

- ❑ **Ex08** - Liste todas as colunas da tabela alunos cuja idade seja maior ou igual a 30, listar apenas as três primeiras linhas encontradas;

id_aluno	nome	data_nascimento	sexo	idade	fone	email
1	Alan Moraes	1989-09-04	M	36	(87) 97096-2547	alan.moraes43@gmail.com
2	Alessandro Soares	1989-10-04	M	36	(87) 96587-6600	alessandro.soares20@hotmail.c...
3	Alexander Peres	1989-08-22	M	36	(81) 94401-2285	alexander.peres86@hotmail.com

- ❑ **Ex09** - Liste o nome e a data de nascimento da tabela alunos que possuam idade entre 20 e 22;

nome	data_nascimento
Cinthia Ade	2003-10-02
Clara Menegon	2003-11-13
Deborah Laura	2003-10-02
Denise Cda	2003-11-13
Mariana Yuriko	2006-06-10
Mariana Sda	2003-11-11
Marilia Shimada	2004-02-20
Marina Thais	2004-01-07
Paula Cde	2004-02-11
Luiz Carolina	2004-01-06



Vamos Exercitar - SELECT

- ❑ **Ex10** - Liste o nome e o telefone dos alunos que tem o nome iniciando pela letra “O”

nome	fone
Okaue Mde	(87) 93462-2845
Olivia Miranda	(81) 94679-4867
Orlando Ade	(81) 98988-3147
Osvaldo Matheus	(87) 99423-1798
Otavio Strata	(87) 99207-6874
Otavio Pimentel	(87) 97127-8388

- ❑ **Ex11** - Liste o nome, email e idade das pessoas que nasceram antes do ano 2000 traga apenas 5 registros;

nome	idade	email
Alan Moraes	36	alan.moraes43@gmail.com
Alessandro Soares	36	alessandro.soares20@hotmail.c...
Alexander Peres	36	alexander.peres86@hotmail.com
Alexandre Sayuri	37	alexandre.sayuri34@gmail.com
Alexandre Moraes	37	alexandre.moraes60@outlook.c...



Vamos Exercitar - SELECT

- ❑ **Ex12** - Liste o nome e o email das pessoas que tem o email de uma conta que **NÃO** possui “@gmail” liste apenas os primeiros 5 registros encontrados.

nome	email
Alessandro Soares	alessandro.soares20@hotmail.com
Alexander Peres	alexander.peres86@hotmail.com
Alexandre Moraes	alexandre.moraes60@outlook.com
Alexandre Bueno	alexandre.bueno37@hotmail.com
Alexis Vannucchi	alexis.vannucchi73@outlook.com

- ❑ **Ex13** - Liste o nome dos alunos que possuam um sobrenome “Oliveira”.

nome
Julia Oliveira
Marina Oliveira
Natalia Oliveira
Eduardo Oliveira



Vamos Exercitar - SELECT

❑ Ex14 - Liste o nome, email e o fone dos alunos que não informaram o fone

nome	email	fone
Alan Moraes	alan.moraes43@gmail.com	NULL
Alexandre Sayuri	alexandre.sayuri34@gmail.com	NULL
Gabriela Carolina	gabriela.carolina13@hotmail.com	NULL



Funções de Agregação

Função	Finalidade	Aplicação	Exemplo
MAX()	Retorna o maior valor de um conjunto de valores	Pode ser usada em colunas de dados numéricos, de texto (para encontrar o valor que vem por último em ordem alfabética) ou de data (para encontrar a data mais recente).	Em uma tabela de vendas, MAX(valor_venda) retornaria o valor da maior venda já realizada.
MIN()	Retorna o menor valor de um conjunto de valores	Semelhante à MAX() , pode ser usada em colunas de dados numéricos, de texto (para encontrar o valor que vem primeiro em ordem alfabética) ou de data (para encontrar a data mais antiga).	Em uma tabela de produtos, MIN(preço) retornaria o preço do produto mais barato
COUNT()	Conta o número de linhas que satisfazem uma determinada condição	• COUNT(*): Conta todas as linhas da tabela. • COUNT(nome_coluna): Conta o número de valores não nulos na coluna especificada. • COUNT(DISTINCT nome_coluna): Conta o número de valores únicos e não nulos na coluna.	Em uma tabela de clientes, COUNT(*) retornaria o número total de clientes. Se você quisesse saber quantos clientes diferentes compraram um produto específico, usaria COUNT(DISTINCT id_cliente) .
AVG()	Calcula a média aritmética de um conjunto de valores numéricos	Soma todos os valores e divide pela quantidade de valores não nulos	Em uma tabela de notas de alunos, AVG(nota) retornaria a nota média da turma.
SUM	Soma todos os valores em uma coluna numérica	Utilizada para cálculos financeiros, análise estatística e acompanhamento de inventário	Em uma tabela de pedidos, SUM(valor_total) retornaria o valor total de todas as vendas.



MAX() e MIN()

```
SELECT MIN(column_name), MAX(column_name),  
FROM table_name  
WHERE condition;
```

Ex: `SELECT MIN(nome), MAX(idade) FROM pessoas;`

cpf	nome	telefone	idade
111	Rogério Aguiar	(81) 12345	50
222	Carlos Silveira	(81) 98765	30
333	Fátima Bezerra	(81) 17166	18

MIN(nome)	MAX(idade)
Carlos Silveira	50

*** ERRO***

```
SELECT nome, MIN(idade) FROM pessoas;
```

O MySQL não sabe qual **nome** deve associar ao valor único de **MIN(idade)**, pois pode haver várias pessoas com a mesma idade mínima.

```
SELECT nome, cpf, idade FROM pessoas  
WHERE idade = MIN(idade);
```

Nenhuma função de agregação pode ser usada diretamente na cláusula WHERE. Para selecionar as informações da pessoa (ou pessoas) com a idade mínima, você deve usar uma **subconsulta** (ou **subquery**). Essa **subconsulta** primeiro encontra o valor mínimo da idade e depois a consulta principal usa esse valor para filtrar os resultados.

```
SELECT cpf, nome, idade FROM pessoas  
WHERE idade = (SELECT MIN(idade) FROM alunos);
```

cpf	nome	idade
333	Fátima Bezerra	18



Count(), AVG() e SUM()

cpf	nome	telefone	idade
111	Rogério Aguiar	(81) 12345	50
222	Carlos Silveira	(81) 98765	30
333	Fátima Bezerra	(81) 17166	18

COUNT(): retorna o quantidade de linhas especificadas na condição

AVG(): retorna a média dos valores de uma coluna numérica

SUM(): retorna a soma dos valores de uma coluna numérica

```
SELECT COUNT(column_name), AVG(column_name), SUM(column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

Ex:

```
SELECT COUNT(cpf), SUM(idade), AVG(idade)
FROM pessoas;
```

COUNT(cpf)	SUM(idade)	AVG(idade)
3	98	32.6667

```
SELECT COUNT(cpf) QTD, SUM(idade) SOMA, AVG(idade) MEDIA
FROM pessoas;
```

QTD	SOMA	MEDIA
3	98	32.6667

```
SELECT COUNT(*) QTD, SUM(idade) SOMA, AVG(idade) MEDIA
FROM pessoas
WHERE idade >= 20;
```

Obs.: O COUNT() pode ser utilizado com “*”

QTD	SOMA	MEDIA
2	80	40.0000



Manipulação da Datas

Função	Comando	Exemplo
Data Corrente	CURRENT_DATE	2022-09-07
Formatar Data	DATE_FORMAT("2022-09-07", "%d/%m/%Y")	07/09/2022
Dia	DAY(campo_data)	DAY('2022-09-07') → 7
Mês	MONTH(campo_data)	MONTH('2022-09-07') → 9
Ano	YEAR(campo_data)	YEAR('2022-09-07') → 2022

personas
cpf VARCHAR(14)
nome VARCHAR(30)
idade INT
datadenascimento DA...
fone VARCHAR(15)
email VARCHAR(100)
Indexes
PRIMARY

Ex: **SELECT** DATE_FORMAT(CURRENT_DATE,"%d/%m/%Y") Data,
DAY(CURRENT_DATE) Dia,
MONTH(CURRENT_DATE) 'Mês',
YEAR(CURRENT_DATE) ANO;

Data	Dia	Mês	ANO
27/10/2025	27	10	2025

Aniversariantes do Mês 6

SELECT Nome, DAY(data_nascimento) Dia
FROM pessoas
WHERE MONTH(datadenascimento) = 6
ORDER BY DIA;

Nome	Dia
Luiza Trajano	14
Janguier Diniz	15
Hélio Figueredo	16
Cláudia Alencar	17

nome	datadenascimento
Rogério Aguiar	2000-12-14
Maria da Luz	2000-12-15
Fátima Bernardes	2000-12-16
Carlos Alencar	2000-12-17
Jorge Mateus	2000-12-18
Rafaela guimarãrs	2000-12-19
Luiza Trajano	1995-06-14
Janguier Diniz	1995-06-15
Hélio Figueredo	1995-06-16
Cláudia Alencar	1995-06-17



Vamos Exercitar – Funções de Agrupamento

- ❑ EX15 - Liste quantos alunos nasceram após o ano 2000

QTD
73

- ❑ EX16 - Liste a média das idades dos alunos com idade maior ou igual a 30

Média
33.3229

- ❑ EX17 - Lista a menor idade dos alunos que nasceram antes de 2000

Menor Idade
26



Vamos Exercitar – Funções de Agrupamento

- ❑ EX18 - Liste nome e o fone dos alunos que tem idade igual a menor idade do grupo

nome	fone	idade
Mariana Yuriko	(81) 94923-1731	20

- ❑ EX19 - Liste o nome dos alunos que tem o dígrafo “rr” em seu nome.

nome
Ana Ferreira
Ana Rerreira
Fernanda Arruda
Lee Barros
Monique Marretto
Nadia Ferreira
Andre Terra
Andre Ferreira
Daniel Ferreira
Estevao Ferreira
Fausto Ferreira
Fernando Ferrari
Guilherme Ferreira
Gustavo Beserra
Jefferson Barreto
Jonathan Carret...
Marcus Ferreira
Pedro Ferreira
Plinio Correa

- ❑ EX20 - Liste a menor idade, a maior idade e a média das idades dos alunos cadastrados

Mínima	Máxima	Média
20	38	30.8876



Vamos Exercitar – Funções de Agrupamento

- ❑ **EX21** - Liste o nome e idade dos alunos que tem idade maior que a média das idades do grupo

nome	idade
Amineres Khae	30
Ana Arena	30
Ana Azevedo	30
Andressa Esperidiao	29
Anna Andrade	29

.....

- ❑ **EX22** - Liste o nome ou nomes dos alunos cujo comprimento seja igual ao comprimento do maior nome cadastrado

nome
Alexsandro Angeluzzi

Rafael Mariani	29
Rafael Goncalves	25
Rafael Icaro	27
Rafael Morais	26
Rafael Fonseca	29
Ranie Stella	25
Raniere Duarte	27

351 registros



GROUP BY(grupado por)

A instrução **GROUP BY** é frequentemente usada com funções agregadas (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar o conjunto de resultados por uma ou mais colunas.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

cpf	nome	telefone	idade	UF
111	Rogério Aguiar	(81) 12345	50	PE
222	Carlos Silveira	(81) 98765	18	PB
333	Fátima Bezerra	(81) 17166	18	PE

Ex:

```
SELECT UF, AVG(IDADE) 'Média Idade por Estado', MIN(Idade) 'Menor Idade'
FROM pessoas
GROUP BY UF;
```

UF	Média Idade por Estado	Menor Idade
PB	18.0000	18
PE	34.0000	18



A cláusula **HAVING** utilizada para colocar condições para os resultados das funções agregadas é equivalente ao WHERE para o SELECT.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Ex: `SELECT UF, AVG(IDADE) 'Média Idade por Estado',
MIN(Idade) 'Menor Idade'
FROM pessoas
GROUP BY UF;`

```
SELECT UF, AVG(IDADE) 'Média Idade por Estado',  
MIN(Idade) 'Menor Idade'  
FROM pessoas  
GROUP BY UF  
HAVING AVG(IDADE) > 30;
```

cpf	nome	telefone	idade	UF
111	Rogério Aguiar	(81) 12345	50	PE
222	Carlos Silveira	(81) 98765	18	PB
333	Fátima Bezerra	(81) 17166	18	PE

UF	Média Idade por Estado	Menor Idade
PB	18.0000	18
PE	34.0000	18

UF	Média Idade por Estado	Menor Idade
PE	34.0000	18



Vamos Exercitar – Funções de Agrupamento

- ❑ **Ex23** - Liste a média das idades por ano de todos os alunos(ano, media das idades), dos alunos que nasceram após o ano 2000.

ANO	Média Idade
2000	25.6383
2001	24.6875
2002	23.7143
2003	22.6667
2004	22.0000
2006	20.0000

- ❑ **Ex24** - Liste a quantidade de pessoas agrupadas por idade(idade, qtd), ordene por idade

idade	qtd
20	1
22	9
23	16
24	25
25	39
26	44
27	45
28	43
29	59
30	70
31	64
32	66
33	64
34	73
35	65
36	77
37	25
38	7

Prof. ME Rogério Aguiar C. Teixeira

+55 81 9.9247-4667

@prof_rogerio_aguiar

@treina_recife

rogerio.aguiar@treinarecife.com.br

<http://br.linkedin.com/pub/rogerio-aguiar->

[teixeira/23/634/28a](http://br.linkedin.com/pub/rogerio-aguiar-teixeira/23/634/28a)

Obrigado!